Flight Delay Prediction: NYC Flights Dataset (2013)

Aarsh Bhatnagar, Anurag Kumar Tiwari, Nav Prabhat Singh, Ashish Kumar Yadav, Shivansh Tiwari
Team T029 – Section E
Computer Science and Artificial Intelligence Student Project
Enrollment Nos.: 2401010007, 2401020009, 2401010293, 2401020015, 2401020108

***Abstract*—This project develops and evaluates a binary classifier for identifying flights that arrive at least 15 minutes late using the 2013 New York City flights dataset for JFK, LaGuardia (LGA), and Newark Liberty (EWR). The principal design constraint is operational realism: predictions are defined to be made at 6:00 PM on the day before scheduled departure, so predictors must reflect information available by that cutoff. Of 336,776 source records, 9,430 have no observed arrival-delay label and are excluded, leaving 327,346 labelled flights. The workflow combines exploratory data analysis, leakage auditing, calendar and schedule feature engineering, preprocessing pipelines, chronological training/validation/testing, and a validation-only decision-threshold choice. A prior-probability dummy classifier, regularized Logistic Regression, and XGBoost are compared. In the recorded run, Logistic Regression achieves the strongest validation Average Precision (0.2348 versus 0.2301 for XGBoost) and is selected. On the November–December holdout, it obtains Average Precision 0.3359, ROC-AUC 0.6070, precision 0.3415, recall 0.4456, and F1-score 0.3867 at threshold 0.2611. These results indicate measurable but limited ranking ability; they do not establish deployment readiness. The report documents the theoretical reasoning behind each method, defines key terms, interprets class-prevalence drift, and identifies limitations including absent weather observations and exclusion of flights without observed arrival delay.**

***Index Terms*—flight delay prediction, binary classification, Logistic Regression, XGBoost, temporal validation, data leakage, class imbalance, feature engineering**

I. Introduction

Flight delays create consequences for passengers, airline schedules, airport resource allocation, and downstream connections. A risk estimate made before a flight operates could support planning, but only if the estimate is based on information available at the time the decision is made. Historical operational datasets contain both schedule information and outcomes observed after take-off; mixing these two types of fields can make a model appear accurate for the wrong reason.

This project studies whether a flight departing from JFK, LGA, or EWR in 2013 will arrive at least 15 minutes late. The threshold yields a binary classification problem rather than a prediction of exact delay minutes. The dataset contains 336,776 records, with 327,346 retained after records with missing arrival-delay labels are excluded. The overall delayed-flight prevalence in the exploratory analysis is 24.47% [1], [2], [9].

The project emphasizes three principles: (1) define the prediction cutoff before selecting features; (2) preserve chronology when evaluating future-period predictions; and (3) compare a simple baseline with a more flexible model using metrics suited to a less prevalent positive class. This approach follows research reviews noting that model complexity alone does not guarantee a better flight-delay predictor and that robust variable selection and evaluation design are central to meaningful comparisons [3], [4].

II. Problem Definition and Objectives

Let D_i denote the recorded arrival delay in minutes for flight i, defined by the dataset's actual-versus-scheduled arrival times. For records with an observed value, the label is

$$y_i = \begin{cases} 1, & D_i \geq 15, \\ 0, & D_i < 15. \end{cases} \tag{1}$$

The task is to estimate $P(y_i = 1 \mid x_i)$ using a feature vector x_i available at the prediction cutoff. The cutoff is fixed at 6:00 PM on the day before the scheduled flight. Accordingly, realized departure time, departure delay, actual arrival time, air time, and arrival delay are prohibited as model inputs. Arrival delay is used only to construct the label in (1).

The objectives are to: (a) characterize missingness and the distribution of delay labels; (b) construct schedule/calendar features consistent with the cutoff; (c) compare a prior baseline, Logistic Regression, and XGBoost; (d) choose the model and classification threshold using validation data only; and (e) estimate performance once on later, held-out dates. The report describes the recorded Phase 1 machine-learning workflow. It does not claim that a deep-learning model has been trained or evaluated in the reviewed implementation.

III. Related Work and Theoretical Rationale

Flight-delay prediction has been investigated as classification, regression, and spatiotemporal forecasting. A recent systematic review identifies growing use of machine-learning and deep-learning methods while noting continuing gaps in uncertainty modelling, external data integration, and deployment realism [3]. A separate benchmarking meta-analysis emphasizes that a more complex algorithm does not automatically produce superior results; feature validity and experimental design remain important [4]. These findings motivate a controlled model comparison rather than an assumption that boosting must win.

A. Why Logistic Regression is a baseline

Logistic Regression estimates the conditional probability of the positive class through

$$p_i = P(y_i = 1 \mid x_i) = \sigma(\beta_0 + \boldsymbol{\beta}^{\mathsf{T}} x_i), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{2}$$

With L2 regularization, coefficient magnitudes are penalized to reduce sensitivity to large or redundant coefficients. The model is computationally practical for one-hot encoded tabular features and provides an informative benchmark. Its linear log-odds form may under-represent interactions or non-linear relationships unless features encode them explicitly [8].

B. Why XGBoost is compared

XGBoost builds an additive ensemble of trees, where later trees improve the current prediction by fitting residual structure related to the chosen loss. This provides a mechanism for learning non-linear effects and feature interactions that are not explicitly represented by a linear classifier [5]. The tested configuration uses 400 estimators, maximum tree depth 5, learning rate 0.05, row subsampling 0.8, and column subsampling 0.8. These values define a candidate model, not a guarantee of superiority; the validation period determines whether the added flexibility is useful here.

C. Why a dummy baseline is necessary

A `DummyClassifier` using the training class prior does not learn from the predictors. It provides a minimum reference for determining whether feature-based models rank delayed flights more effectively than the class prevalence alone. On validation, the prior baseline Average Precision equals approximately 0.1570, the validation positive-class prevalence. A candidate model that fails to beat this reference has not demonstrated ranking information beyond the base rate.

IV. Dataset and Exploratory Data Analysis

A. Dataset profile and target cleaning

The source file contains 336,776 records and 21 columns from flights departing JFK, LGA, and EWR in 2013 [1], [2], [9]. The EDA notebook reports 9,430 missing values for `arr_delay`. Since the label cannot be calculated for these records, they are excluded from this binary classification experiment. The final labelled sample contains 327,346 flights, or approximately 97.2% of the source records. The resulting task does not classify cancellations or diversions and may carry selection bias because the excluded records are not a random sample of all scheduled flights.

The overall positive-class prevalence is reported as 24.47%. Therefore the negative class is more common, and a classifier that predicts every flight as not delayed would achieve a seemingly high accuracy while detecting no delayed flights. This motivates reporting Average Precision and recall alongside threshold-dependent measures. The exploratory analysis is descriptive; associations found in this one-year dataset are not interpreted as causal effects.

B. Patterns identified during EDA

The notebook reports that delay rates tend to increase later in the day: early-morning hours fall below 15% in the displayed analysis, while late-evening hours can exceed 30%. Saturday is the calmest weekday at roughly 17%, while Thursday is the worst at about 28%. Seasonal variation is also visible:

label, 327,346 flights remain. The positive class is defined
east 15 minutes; its prevalence is 24.47%. This class distribu
verage Precision alongside ROC-AUC rather than relying on a

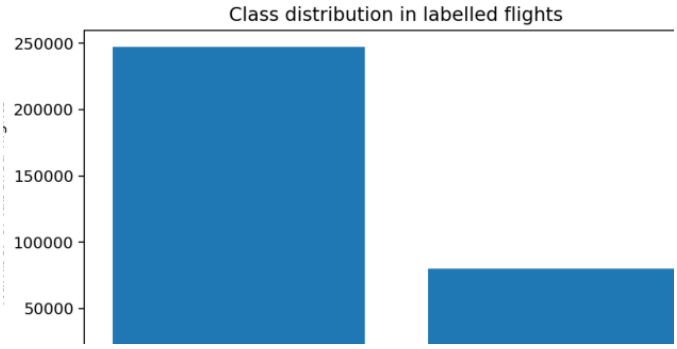


Fig. 1. Class distribution among flights with an observed arrival-delay label.

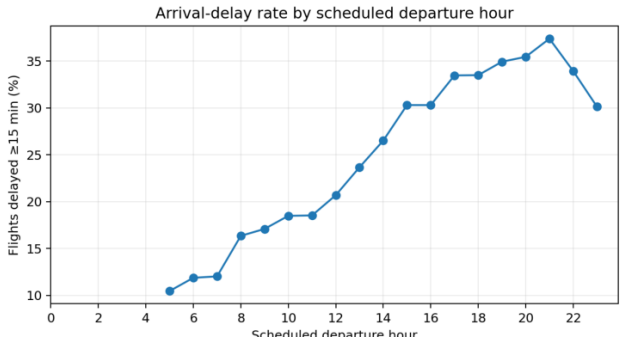


Fig. 2. Observed arrival-delay rate by scheduled departure hour. This is descriptive analysis, not a causal effect.

September is relatively calm (around 14%), whereas summer and December have higher delay rates. These patterns support schedule and calendar features, but they do not prove that the schedule or weekday causes an individual delay.

A univariate leakage audit illustrates why operational availability matters. The EDA notebook reports that `dep_delay` alone yields ROC-AUC approximately 0.895 for the target, and about 79.9% of flights departing 15 or more minutes late subsequently meet the arrival-delay label. However, actual departure delay is known only after departure. It is thus a forbidden feature for the intended prediction time, regardless of its predictive power. The notebook similarly excludes `dep_time`, `arr_time`, and `air_time` from the predictor set.

V. Feature Engineering and Preprocessing

A. Predictor groups

The modelling notebook uses four categorical feature groups: carrier, origin airport, grouped destination, and flight number. Numerical and schedule-derived inputs include distance, scheduled departure minute, scheduled arrival minute, the count of scheduled flights at the same origin and hour, and cyclical representations of scheduled departure hour, scheduled

sis, not a causal effect.

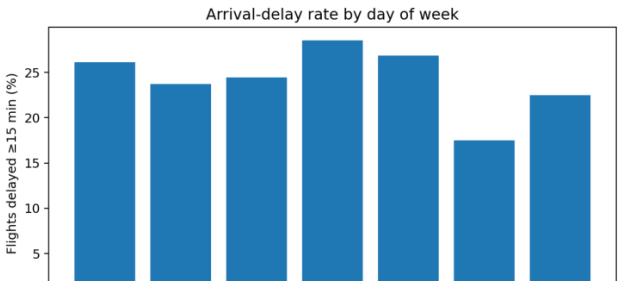


Fig. 3. Observed arrival-delay rate by weekday. Differences may reflect seasonality, schedule composition, or operational conditions.

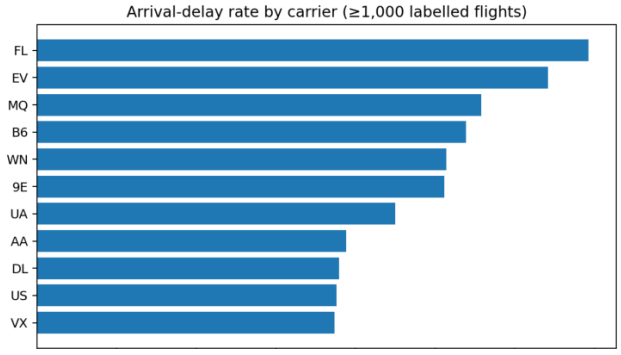


Fig. 4. Observed arrival-delay rate by carrier, restricted to carriers with at least 1,000 labelled flights.

arrival hour, weekday, and day of year. The goal is to encode persistent schedule patterns without importing realized operating outcomes.

Periodic values are represented with sine and cosine transformations:

$$x_{\sin} = \sin\left(\frac{2\pi x}{P}\right), \qquad x_{\cos} = \cos\left(\frac{2\pi x}{P}\right), \tag{3}$$

where P is 24 for hour of day, 7 for weekday, and 365 for day-of-year in 2013. This avoids treating, for example, 23:00 and 00:00 as distant values on a linear number line. Rare destination values are grouped into an `OTHER` category based on the training data to reduce unstable estimates from very small groups.

The `scheduled_origin_hour_count` feature counts scheduled flights in the corresponding origin/hour grouping. It is a schedule-based proxy, not a direct observation of runway congestion, weather, or real-time airport conditions. Its use is defensible only if the relevant schedule is available by the stated 6 PM cutoff and if the count is reconstructed without outcomes or future information. This availability assumption must be preserved if the workflow is reused operationally.

The feature `flight` is retained as a categorical variable in the current allow-list. Because flight numbers may recur across dates, they may encode useful route/service regularity; they may also allow memorization of repeated services. An ablation comparison without this variable is therefore a useful future test of generalization.

TABLE I
CHRONOLOGICAL DATA PARTITIONS

Subset	Rows	Delayed	Purpose
Train (Jan–Aug)	217,727	26.37%	Fit preprocessing and model parameters
Validation (Sep–Oct)	55,628	15.70%	Select model and threshold
Test (Nov–Dec)	53,991	25.83%	Final held-out evaluation

B. Missing values, encoding, and pipeline design

The implementation places preprocessing and each classifier into a scikit-learn pipeline. Numeric values are imputed with the training median and standardized using training-set statistics. Categorical values are imputed with the training most frequent value and one-hot encoded; the encoder is configured to ignore unseen categories and to group infrequent categories at a minimum frequency of 20. This design applies the same learned transformations at prediction time and reduces the chance that validation or test data influence imputation, scaling, or category encoding [8].

The train/validation/test split is created before fitting model-specific preprocessing. A check in the preprocessing notebook reports that the leakage allow-list check passed and that the output matrices contain 16 preprocessed model features before one-hot expansion. Missing target labels are treated differently from missing predictors: records without observed `arr_delay` are excluded, while missing predictor values are handled within the pipeline.

VI. Experimental Protocol

A. Chronological partitioning

The data are split by calendar month rather than randomly shuffled. This is important because the intended task is prediction for future flights and because patterns can change over the course of a year. The recorded partitions are shown in Table I.

The validation positive-class prevalence is about 10.7 percentage points lower than training prevalence and 10.1 points lower than test prevalence. This is material temporal variation. Since Average Precision depends on positive-class prevalence, raw AP values from different splits should not be compared without also reporting the corresponding base rates. Chronological splitting avoids temporal mixing but does not eliminate distribution shift; rolling-origin evaluation would provide a stronger robustness analysis.

B. Model candidates and selection rule

The experiment compares the prior-probability dummy baseline, Logistic Regression, and XGBoost. Candidate comparison is based on Average Precision on the validation set. Logistic Regression reaches validation AP of 0.2348 and ROC-AUC of 0.6446, compared with AP 0.2301 and ROC-AUC 0.6368 for

TABLE II
VALIDATION MODEL COMPARISON

Model	Average Precision	ROC-AUC
Prior dummy baseline	0.1570	0.5000
Logistic Regression	**0.2348**	**0.6446**
XGBoost	0.2301	0.6368

XGBoost. Therefore Logistic Regression is selected for final evaluation in the recorded run. This is a small validation AP advantage, not evidence that Logistic Regression universally outperforms boosted trees.

The classification threshold is chosen only on validation predictions by maximizing F1. The resulting threshold is 0.2611, below the conventional 0.5 cutoff. The validation precision, recall, and F1 at this selected operating point are 0.2261, 0.5888, and 0.3268, respectively. The threshold is then frozen before evaluating the November–December test set. Optimizing a threshold on test labels would leak information from the holdout and would make the reported test metrics optimistic.

C. Evaluation metrics

Average Precision (the reported PR-AUC-style score) is emphasized because delayed flights are the less prevalent class. It summarizes precision across recall increments and has a simple constant-score baseline equal to the positive prevalence [7], [8]. ROC-AUC provides a threshold-independent view of ranking discrimination [6]. Precision, recall, and F1 depend on the selected threshold and expose the false-positive/false-negative trade-off. Accuracy is included only as a secondary diagnostic. For a consistent split, PR-AUC lift is computed as

$$\text{Lift}_{AP} = \frac{\text{Average Precision}}{\text{Positive prevalence}}. \tag{4}$$

This lift compares the model with the constant-score precision-recall baseline; AP and prevalence must come from the same subset.

VII. RESULTS

A. Validation comparison

Table II summarizes the recorded validation results. The prior baseline reflects only class prevalence; both learned classifiers improve AP over that reference. Logistic Regression is selected under the predeclared AP criterion.

The learned models show only moderate validation ranking performance. The difference between Logistic Regression and XGBoost is modest, suggesting that the engineered schedule/calendar variables allow a regularized linear model to capture some signal without the additional flexibility of boosted trees. This does not mean non-linear models are unsuitable; rather, the measured validation criterion did not justify selecting XGBoost in this run.

TABLE III
SELECTED MODEL ON THE HELD-OUT TEST SET

Metric	Test result
Selected model	Logistic Regression
Decision threshold	0.2611
Average Precision	0.3359
ROC-AUC	0.6070
Precision	0.3415
Recall	0.4456
F1-score	0.3867
Accuracy	0.6349
Positive-class prevalence	0.2583
AP lift over prevalence	1.30×

B. Held-out test performance

Table III reports results from the selected Logistic Regression model on the later November–December period, using the validation-selected threshold of 0.2611. The test prevalence is 25.83%.

The AP lift is approximately $0.3359/0.2583 = 1.30$, indicating that the ranking is better than a constant-score baseline, but not by a large margin. The ROC-AUC of 0.6070 also indicates limited separation between delayed and non-delayed flights on the later holdout. The selected threshold yields recall of 0.4456, meaning fewer than half of delayed flights are identified at this operating point; precision of 0.3415 means that many flights flagged as delayed are not delayed by the target definition.

The test accuracy of 0.6349 is lower than the 0.7417 accuracy of a trivial rule that labels every test flight as not delayed, derived from the test prevalence of 0.2583. This is not concealed by the report: the operating threshold was selected to balance precision and recall through F1, not to maximize accuracy. Whether this trade-off is useful depends on operational costs. If false alerts are expensive, precision may need to be constrained; if missing a delay is more costly, recall may be prioritized. The current run does not establish a business cost function, so F1 is a convenient experimental threshold rule rather than a claim of operational optimality.

C. Feature reliance

The model-application notebook calculates permutation importance on validation Average Precision. It identifies `hour_sin` and `carrier` among the most predictive features. This is consistent with EDA indicating that delay probability varies by scheduled time and airline. Permutation importance is a model- and dataset-specific measure of predictive reliance; it does not establish that these features cause delays. Correlated inputs can also share importance, and importance rankings may change under temporal distribution shift.

VIII. DISCUSSION AND THEORETICAL RIGOR

A. Why the evaluation design is central

A random split could place similar time periods and repeated service patterns in both training and testing, producing an evaluation that is easier than forecasting genuinely later flights.

The chronological split is therefore aligned with the intended use case. However, a single train/validation/test division is still sensitive to the particular months selected. The 15.70% validation prevalence, contrasted with 26.37% training and 25.83% test prevalence, shows that the model-selection period is atypically calm. This can make model ranking and threshold selection less representative of the final period. Rolling-origin validation and confidence intervals across several forward-looking cutoffs are recommended.

Pipelines are equally important. Imputation, scaling, category grouping, and one-hot encoding must learn parameters from training data only. If these steps are fitted on all records before the temporal split, information about future distributions leaks into development. scikit-learn specifically recommends fitting transformations on training data and applying them unchanged to held-out samples [8].

B. Why the best model is not necessarily the most complex

XGBoost was included because boosted trees can model non-linear patterns and interactions, but the recorded comparison selects Logistic Regression. This is an empirical decision under the specified metric and split. The meta-analysis of flight-delay modelling warns against equating complexity with improvement [4]. Regularized Logistic Regression is also comparatively easier to inspect and cheaper to retrain. Future experiments could test a calibrated XGBoost model, carefully tuned regularization, and rolling-origin validation, but comparisons must preserve the same prediction contract and test protocol.

C. Operational interpretation

The experiment estimates risk, not a causal explanation of a delay. It includes no direct weather observations, so it cannot learn measured storm severity or local meteorological conditions. Schedule-based counts should not be described as real-time congestion. It also does not predict cancellations or diversions because records without arrival-delay labels are removed. Finally, one year of flights from three New York airports is not sufficient to establish generalization to other airports, years, airlines, or regulatory environments.

The recorded performance is appropriate for a research prototype, not for automated operational use. Any deployment decision would require current schedule inputs, monitoring of class prevalence and calibration, explicit alert costs, subgroup analysis, and periodic temporal re-evaluation. The model should provide decision support only after independent validation in the intended operational environment.

IX. Reproducibility and Implementation

The source repository organizes the workflow into exploratory analysis, preprocessing, and model application notebooks [6]. The model-application notebook sets `RANDOM_STATE=42`, uses pandas and NumPy for data handling, matplotlib and seaborn for visual analysis, scikit-learn for preprocessing/model pipelines and metrics, XGBoost for boosted trees, and joblib for model persistence. The preprocessing notebook generates chronological CSV splits; the modelling notebook creates outputs such as the selected model pipeline, test predictions, metric metadata, model-comparison records, and permutation-importance results, depending on the configured output directory.

A reproducible rerun should: (1) obtain the same version of `flights.csv`; (2) restart the notebook runtime; (3) execute the EDA, preprocessing, and model-application notebooks in order; (4) verify the 336,776 raw row count and 327,346 labelled row count; (5) verify the chronological split sizes and prevalence rates; (6) verify that forbidden columns are absent from model inputs; and (7) record package versions, outputs, and the exact commit used for final submission. Notebook output is execution-state-dependent, so the reported metrics should be regenerated after code or data changes.

The implementation is maintained in the project GitHub repository. The two supplied notebooks are available as Colab notebook A and Colab notebook B. Data access and notebook permissions may depend on account and sharing settings.

X. Limitations and Future Work

The limitations that most directly affect validity are:

1) **No weather inputs:** add weather only if timestamped observations were available by the 6 PM cutoff.
2) **Limited scope:** evaluate later years and external airports before generalizing.
3) **Missing target:** model cancellations and diversions as a separate task.
4) **Temporal drift:** use rolling-origin tests and monitor prevalence and calibration.
5) **Feature memorization:** ablate `flight`; verify that the schedule-count feature is available and leakage-free.
6) **Decision policy:** set the threshold using explicit costs or precision/recall requirements, not F1 alone.
7) **Deep learning:** Phase 2 is future work; no deep-learning result is claimed in this report.

XI. Conclusion

This project builds a leakage-conscious binary classifier for arrival delays of at least 15 minutes, using 2013 flights from JFK, LGA, and EWR and a prediction cutoff at 6:00 PM on the previous day. The final modelling sample contains 327,346 labelled flights. The workflow uses cyclic calendar encodings, schedule-based inputs, train-fitted preprocessing pipelines, chronological partitions, a simple class-prior baseline, Logistic Regression, and XGBoost.

In the recorded run, Logistic Regression is selected by validation Average Precision, narrowly ahead of XGBoost. On the November–December holdout it reaches AP 0.3359 and ROC-AUC 0.6070, with F1 0.3867 at a validation-selected threshold of 0.2611. These values indicate limited but non-zero ranking signal, not high-reliability forecasting. The test accuracy is below that of an always-negative rule, illustrating why the target metric and decision costs must be stated rather than relying on a single score. Avoiding post-departure leakage, reporting the base rate of every temporal split, and maintaining

a genuinely untouched test period are central to the validity of the results. Additional weather data, broader temporal validation, calibrated probabilities, explicit operating costs, and a separately validated deep-learning extension are logical next steps.

REFERENCES

[1] M. Sajadi, "Flights," Kaggle dataset, 2013 New York City flight records. [Online]. Available: https://www.kaggle.com/datasets/matinsajadi/flights. Accessed: Oct. 22, 2026.

[2] H. Wickham, "nycflights13: Data about flights departing NYC in 2013," R package, version 0.1, 2014. [Online]. Available: https://github.com/tidyverse/nycflights13.

[3] S. Gharibzadeh, M. M. Pedram, M. Khademi, and A. Salajegheh, "Trends and gaps in machine learning based flight delay prediction in spatiotemporal and data-driven approaches: A systematic review," *Applied Soft Computing*, vol. 202, Part A, Art. no. 115761, Oct. 2026, doi: 10.1016/j.asoc.2026.115761.

[4] H. da Silva Queiróz Júnior, V. Falcão, F. G. F. da Silva, and I. M. T. Bezerra, "Machine learning methods benchmarking for predicting flight delays: An efficiency meta-analysis," *Sustainability*, vol. 17, no. 21, Art. no. 9887, Nov. 2025, doi: 10.3390/su17219887.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[6] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[7] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[8] Scikit-learn Developers, "Common pitfalls and recommended practices: Data leakage," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html. Accessed: Oct. 22, 2026.

[9] abhi-7-7, "Flight Delay Prediction – NYC Flights (2013)," GitHub repository, Team T029, Section E. [Online]. Available: https://github.com/abhi-7-7/Flight_Delay_Prediction. Accessed: Oct. 22, 2026.